

Generative Adversarial Network

Agenda	Generative Models
	New Loss Function
	GANs and Examples
	Adversarial Examples

Last Time

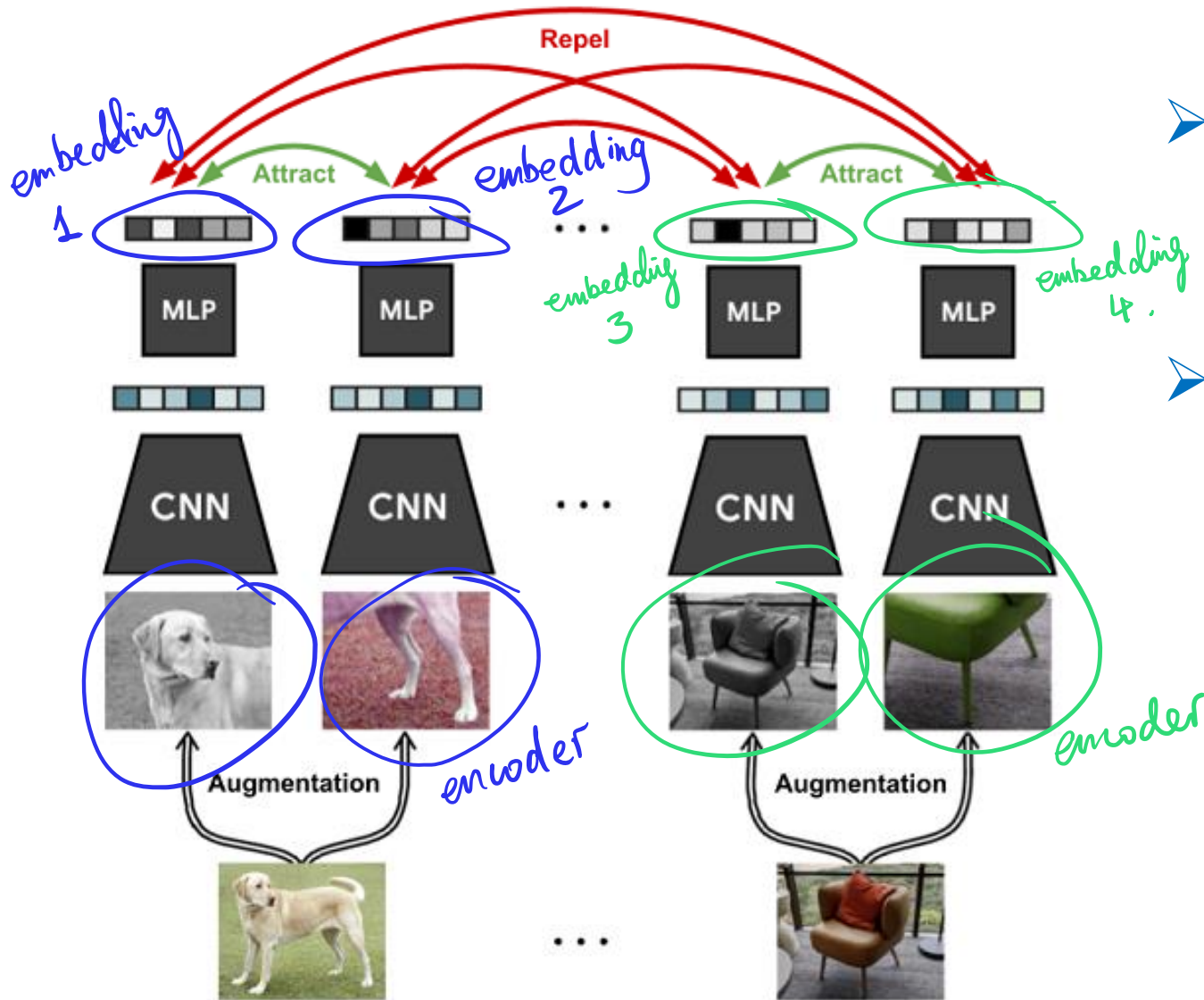
Last Class:

- Unsupervised Learning
 - Semi-Supervised learning
 - Self-Supervised learning
- Upsampling
 - Convolutional Transpose
 - Unpooling
- Autoencoders, VAE

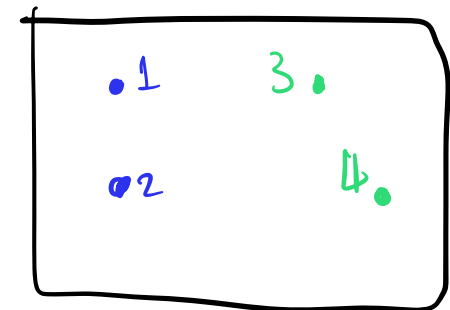
Embedding Space

aka latent space

Recap: Contrastive Learning



- Representation Learning (learning good embeddings)
- Self-supervised learning through data augmentations



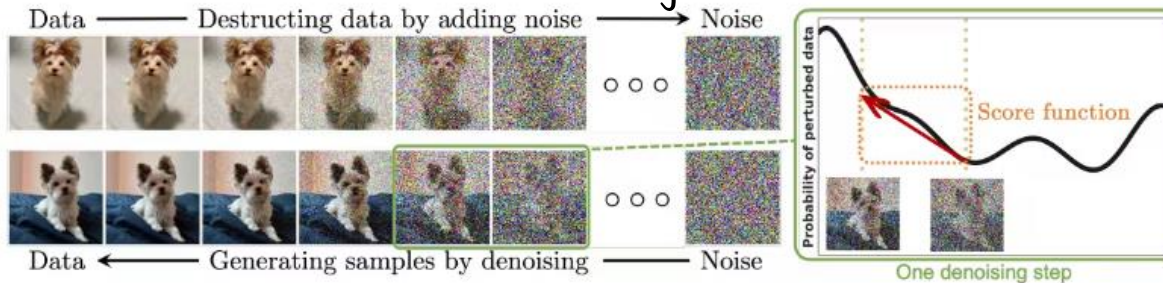
Embedding space.

Source: [Sik-Ho Tsang](#)

embedding from the same image are closer.

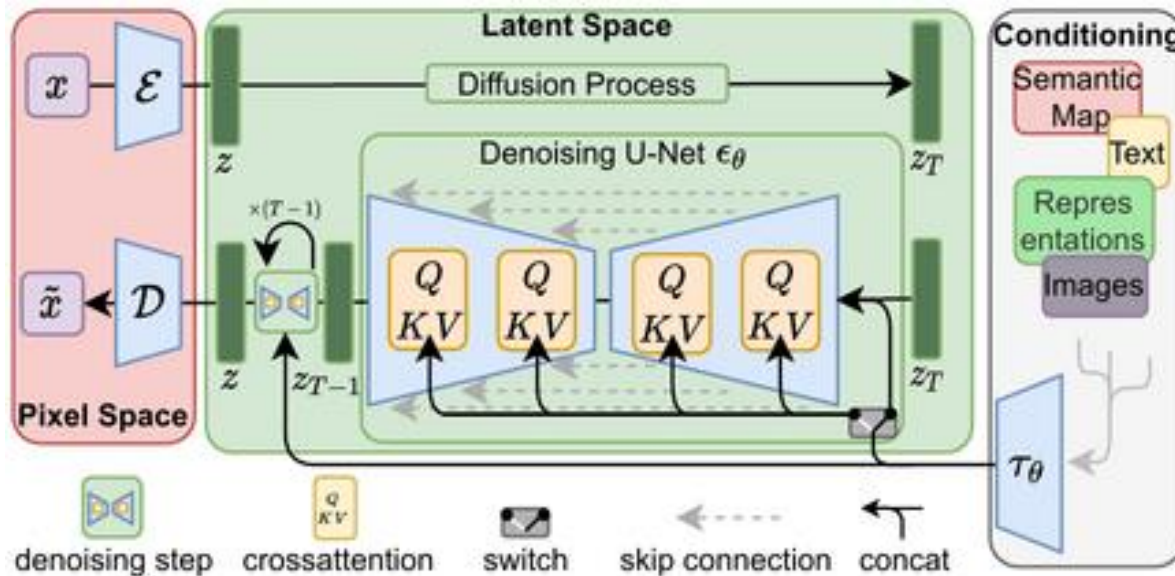
Recap: Diffusion Models

Add noise to the original models
then tell the image what you want and remove noise to give expected image.



➤ U-Net architecture to predict noise and perform denoising

➤ Can be conditioned on text (more on this next time)



Recap: Unsupervised Learning

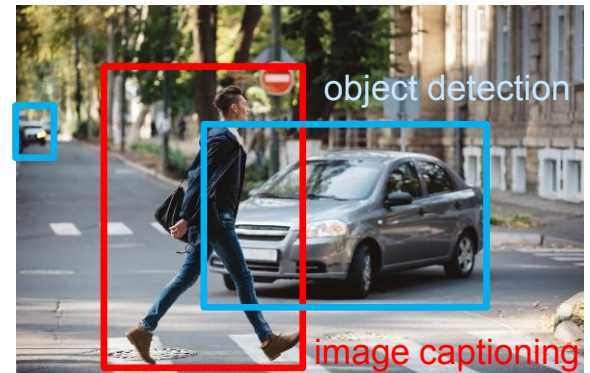
Supervised Learning

- learning a function that maps an input to an output based on example input-output pairs

Disadvantages:

1. Hard to get labels
2. \$\$\$
3. not enough data labels.

classification

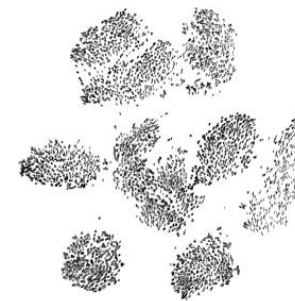
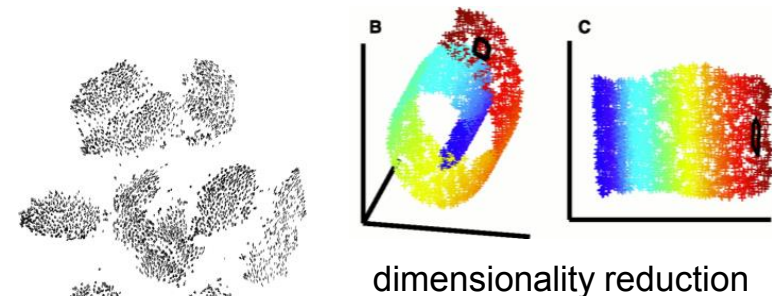


"a man crossing a sidewalk"

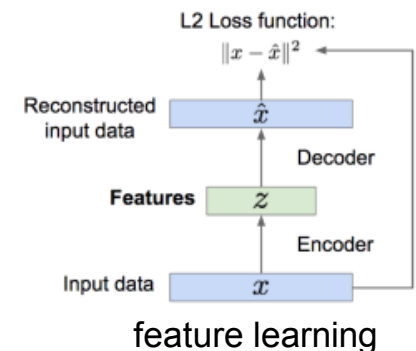
Unsupervised Learning

- learn underlying structure (hidden structure) of unlabeled data

Key difference is the presence or absence of labels



clustering



Discriminative vs Generative Models

Discriminative Model $\Leftrightarrow$ classification model.

- learn a probability distribution $p(y|x)$
learn from label

Generative model

- learn a probability distribution $p(x)$
learn from data

Data: x



Label: y

Car

There is also a Conditional Generative Model which learns $p(x|y)$

Discriminative vs Generative Models

Discriminative Model

- learn a probability distribution $p(y|x)$

Generative Model

- learn a probability distribution $p(x)$

Conditional Generative Mode

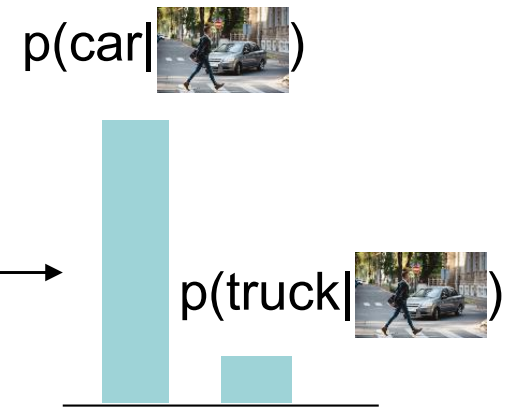
- learns $p(x|y)$

Data: x



Label: y

Car

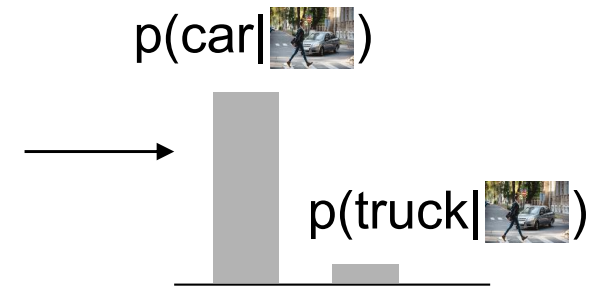


There is a competition between different values of y (probability sums up to 1)

Discriminative vs Generative Models

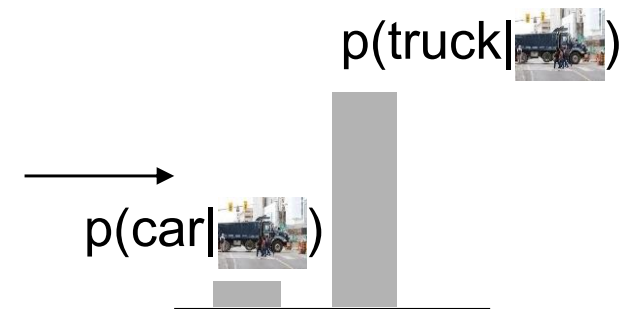
Discriminative Model

- learn a probability distribution $p(y|x)$



Generative Model

- learn a probability distribution $p(x)$



Conditional Generative Mode

- learns $p(x|y)$



Discriminative vs Generative Models

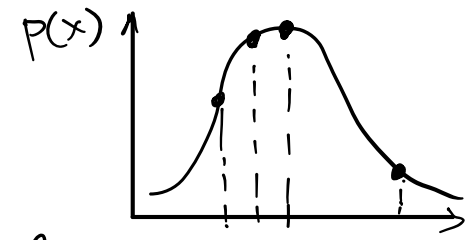
Discriminative Model

- learn a probability distribution $p(y|x)$

→ only tells right/wrong (if belongs to the class).

high

high



lower

$p(\text{horse})$

low

$p(\text{balls})$

Generative Model

- learn a probability distribution $p(x)$

代表如果机
in machine
generate 一个
car, 更可能成
功地生成一个 car
因为 sampling car the most
(被 train 的 data
中 car 最多)

more intelligent than
discriminative
model.

Conditional

Generative Mode

- learns $p(x|y)$

because it needs
to understand
and generate

Generative model: All possible
images compete with each other
for probability density

Compete on x ?



Discriminative vs Generative Models

Discriminative Model

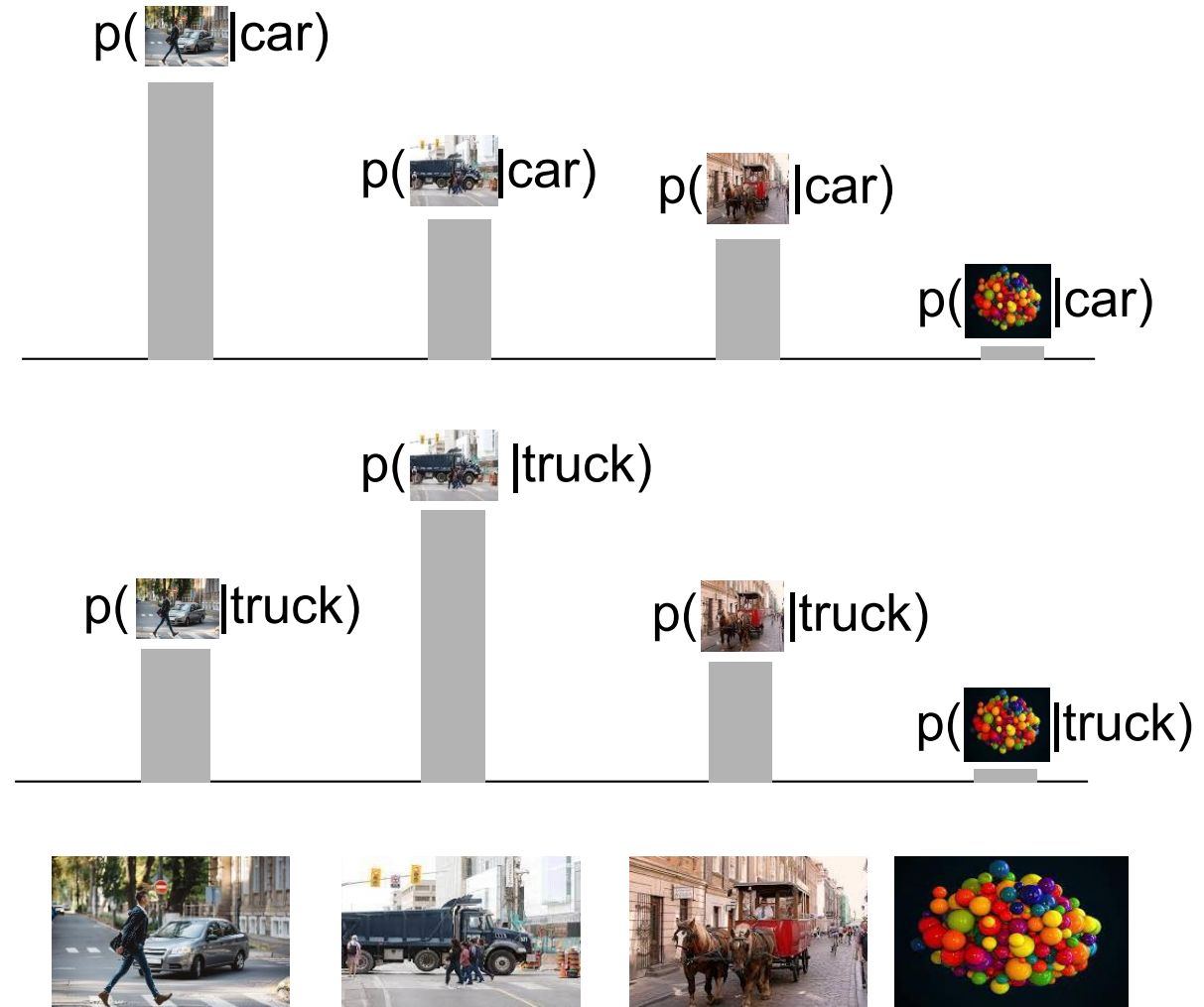
- learn a probability distribution $p(y|x)$

Generative Model

- learn a probability distribution $p(x)$

Conditional Generative Mode

- learns $p(x|y)$



Agenda

Today:

- Generative Adversarial Network
- Conditional GANs and more architectures
- Adversarial Examples

Generative Models

- A generative models learn the structure of data, and can be used to generate new data
- Examples:
 - **Variational autoencoders** to generate images (last lecture)
 - **Diffusion models** (mentioned this lecture)
 - **Recurrent neural network** to generate text (upcoming lectures)
 - **Transformers** (upcoming lectures)

Review Autoencoders – Code!

Example Autoencoder

```
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(16, 32, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 7)
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 7),
            nn.ReLU(),
            nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, 3, stride=2, padding=1, output_padding=1),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x
```

Upsampling
Layers!

Example Autoencoder

```
def train(model, num_epochs=5, batch_size=64, learning_rate=1e-3):
    torch.manual_seed(42)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate, weight_decay=1e-5)
    train_loader = torch.utils.data.DataLoader(mnist_data, batch_size=batch_size, shuffle=True)
    outputs = []
    for epoch in range(num_epochs):
        for data in train_loader:
            img, label = data
            recon = model(img)
            loss = criterion(recon, img)
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
```

Q: How does the loss function typically work in an autoencoder?

Autoencoder Outputs are Blurry

These faces were generated using a variational autoencoder



Q: Which parts of the images are blurry? Why?

The machine is not aligned with people (we) think. We think all pixels are valuable & should encode, decoder all.

But machine doesn't think so.

So return the average pixel

⇒ blurry backgrounds.

Autoencoders Loss

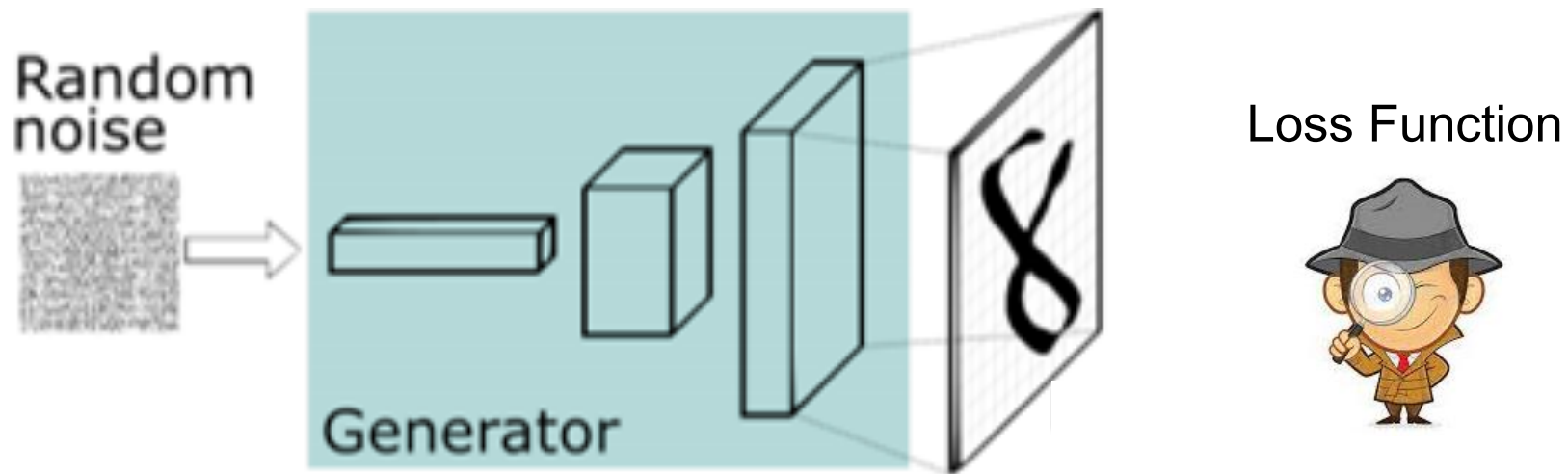
- Blurry images, blurry backgrounds
- Why?
 - Because the loss function used to train an autoencoder is the mean square error loss (MSELoss)
- To minimize the MSE loss, autoencoders predict the “average” pixel – e.g., average the background

Q: Can we use a better loss function?

Generative Adversarial Networks

Generative Adversarial Networks

- First, sample from a simple distribution (similar to what we did with VAE)
- Second, allow the network to effectively learn its own loss function

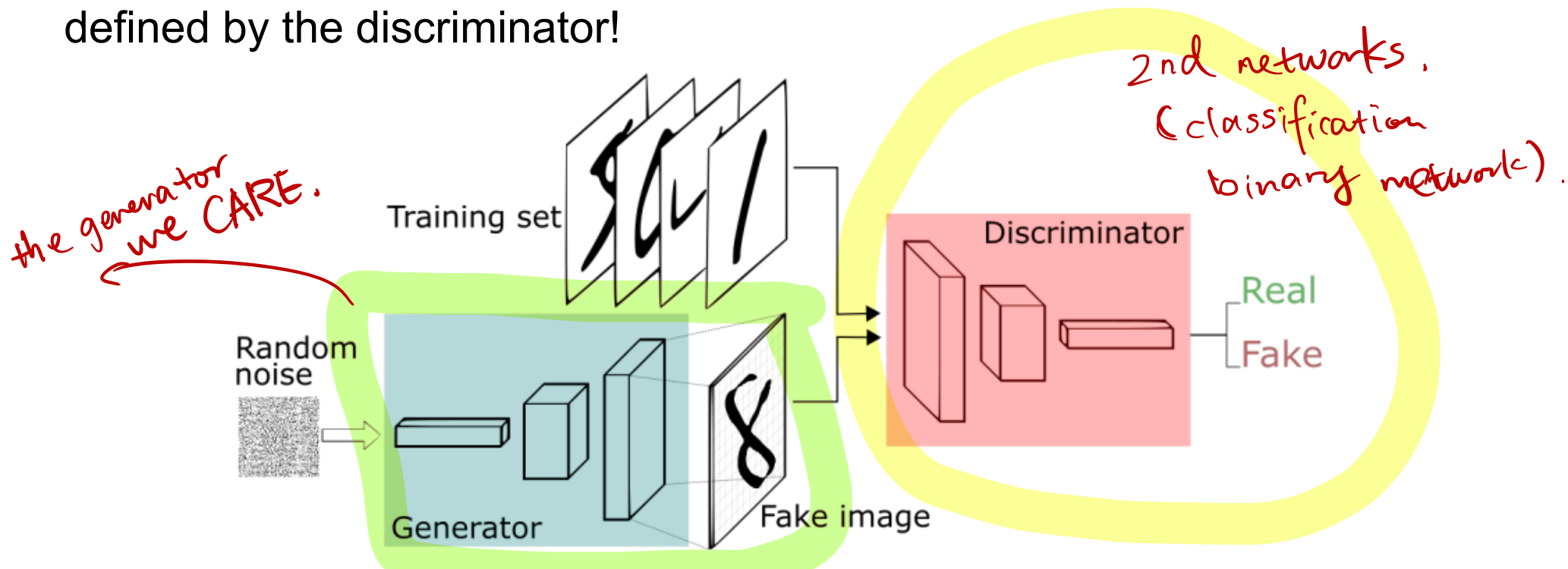


Generative Adversarial Networks

Idea: train two networks

- **Generator network:** try to fool the discriminator by generating real-looking images
- **Discriminator network:** try to distinguish between real and fake images

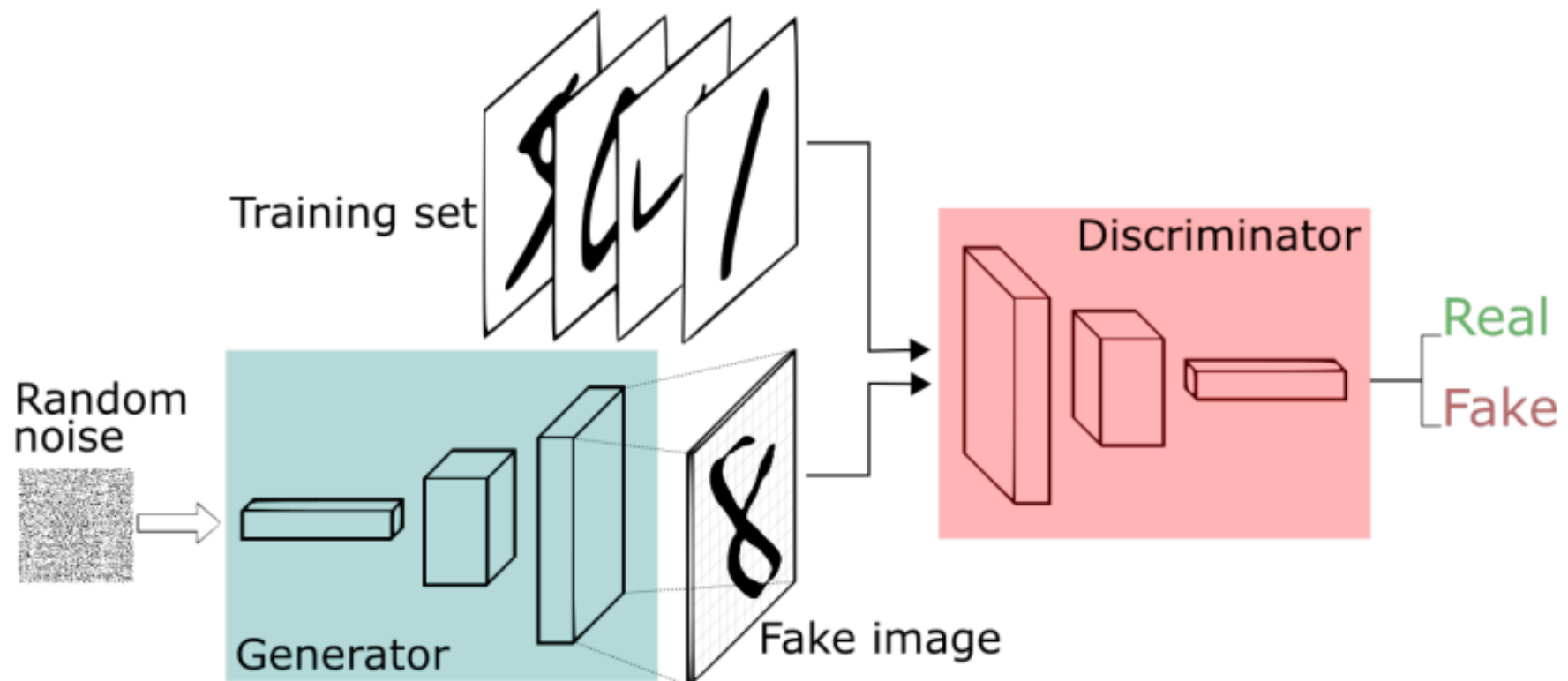
The loss function of the generator (the model we care about) is defined by the discriminator!



Training GANs: two-player game

Play a minmax game:

- the discriminator will try its best to detect real from fake data
- the generator is set to make the discriminator as wrong as possible



Loss function for min-max game

→ want it to mark fake image fake
Tune **discriminator** weights to:

- **maximize** the probability that the...
 - discriminator labels a real image as real
 - discriminator labels a generated image as fake
 - Q: What loss function should we use (discriminator)?

classification
BCE
↓
true/false

Tune **generator** weights to:

- **maximize** the probability that...
 - discriminator labels a generated image as real
 - Q: What loss function should we use (generator)?

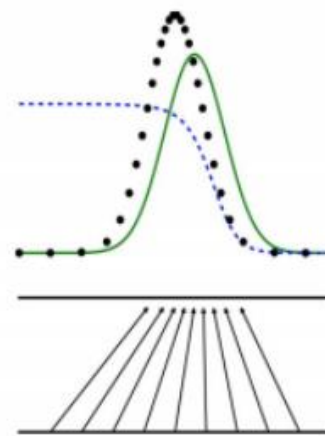
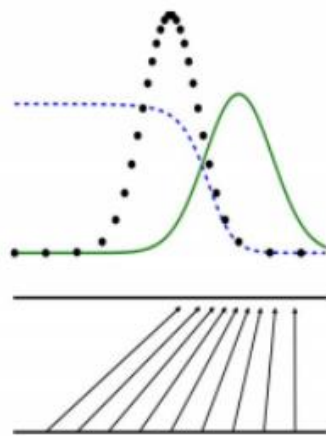
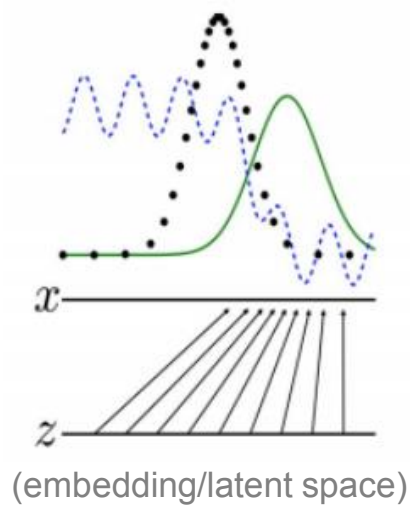
if no need to evaluate output/pred, why need an BCE?

BCE.

Training

Alternate between:

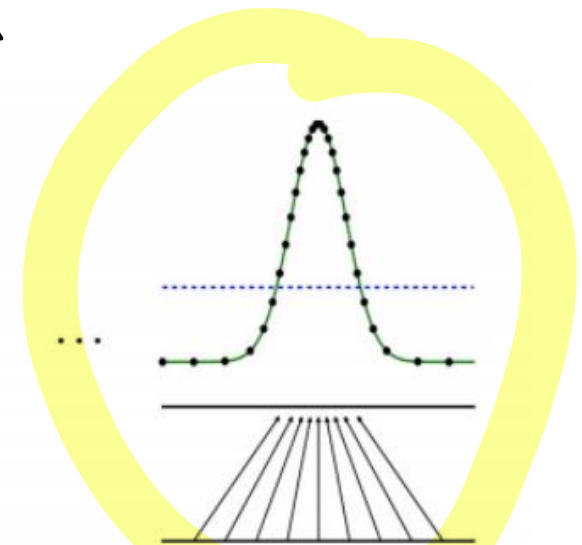
- Training the discriminator
- Training the generator



Legend
Training Data Distribution
Generated Data Distribution
Loss

discriminator model.

generative model



Why 50%?

accuracy of discriminator: 50%

Source: Ian Goodfellow, 2014

Balance Generator and Discriminator

- If the discriminator is too good, then the generator will not learn
 - Remember that we are using the discriminator like a “loss function” for the generator
 - If the discriminator is too good, small changes in the generator weights won't change the discriminator output
 - If small changes in generator weights make no difference, then we can't incrementally improve the generator

Caveat before we look at code

- Can work very well, but **very difficult to train!**
 - Takes a long time to train (a long time before we see progress)
- Difficult to quantify progress
 - Plotting the “training curve” (discriminator/generator loss) doesn’t help much
- To make the GAN train faster, we’ll use:
 - LeakyReLU Activations instead of ReLU
 - Batch Normalization

Why?

GAN: Discriminator

```
class Discriminator(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.model = nn.Sequential(
```

```
            nn.Linear(28*28, 300),
```

```
            nn.LeakyReLU(0.2),
```

```
            nn.Linear(300, 100),
```

```
            nn.LeakyReLU(0.2),
```

```
            nn.Linear(100, 1))
```

```
    def forward(self, x):
```

```
        x = x.view(x.size(0), -1)
```

```
        out = self.model(x)
```

```
        return out.view(x.size(0))
```

like standard CNN

image (MNIST)

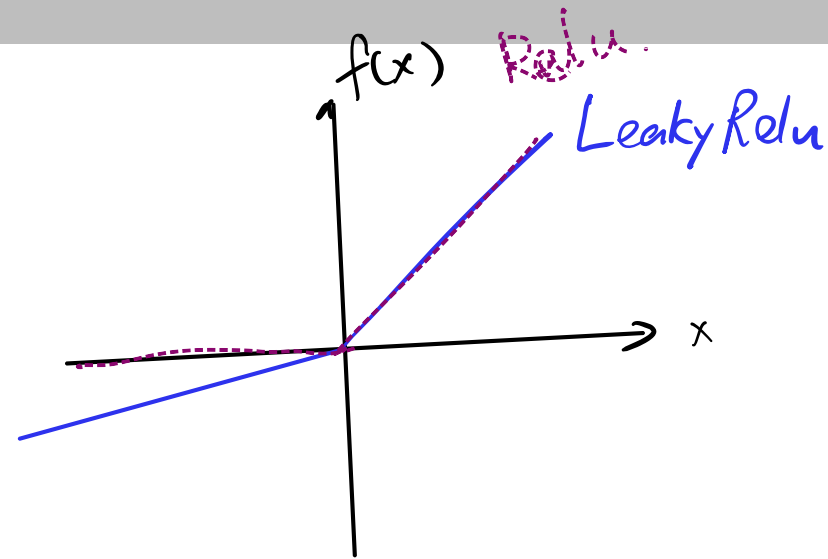
binary output

Leaky ReLU activation

➤ Like a ReLU, but “leaks” data:

➤ $f(x) = x$ if $x \geq 0$

➤ $f(x) = \alpha x$ if $x < 0$



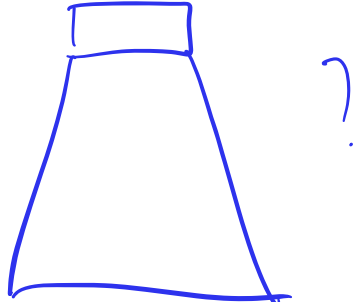
Q: Why would we want to use the leaky ReLU?

➤ Fixes the dead ReLU problem

➤ Less concern with initialization of parameters

GAN: Generator

```
class Generator(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.model = nn.Sequential(  
            nn.Linear(100, 300),  
            nn.LeakyReLU(0.2)  
            nn.Linear(300, 28*28),  
            nn.Sigmoid())  
  
    def forward(self, x):  
        out = self.model(x).view(x.size(0), 1, 28, 28)  
        return out
```



Training the Discriminator

standard binary
classification.

```
#images = batch of images (real)
#batch_size = images.size(0)
noise = torch.randn(batch_size, 100)
fake_images = generator(noise)
inputs = torch.cat([images, fake_images])
labels = torch.cat([torch.zeros(batch_size), # real
                    torch.ones(batch_size)] # fake)
d_outputs = discriminator(inputs)
d_loss = criterion(d_outputs, labels)
```

(Labels: real=0, fake=1) Why fake=1?

因为 input 是 generator 生成的 fake image. (非真实图像)
所以 discriminator 判 false 才是对的

Training the Generator

Q: What is special about the g_loss?

```
noise = torch.randn(batch_size, 100)
fake_images = generator(noise)
outputs = discriminator(fake_images)
```

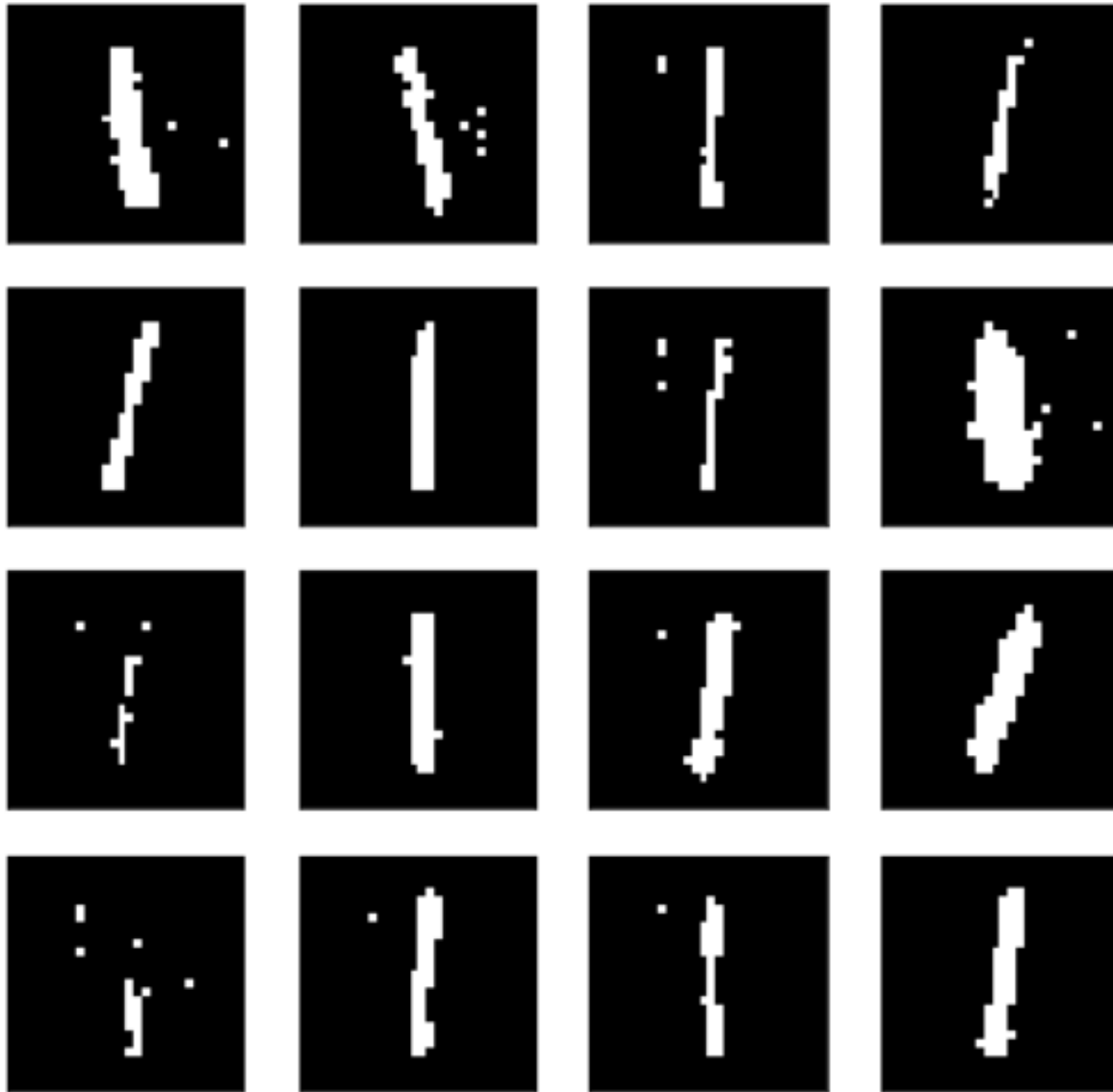
```
g_loss = criterion(outputs, torch.zeros(batch_size))
```

(Labels: real=0, fake=1)

evaluates how wrong the discriminator is.

*→ The more wrong the better
(代表 generator 生成的假图越真)*

Generated MNIST Samples



Q: Why does it
only generate 1s?

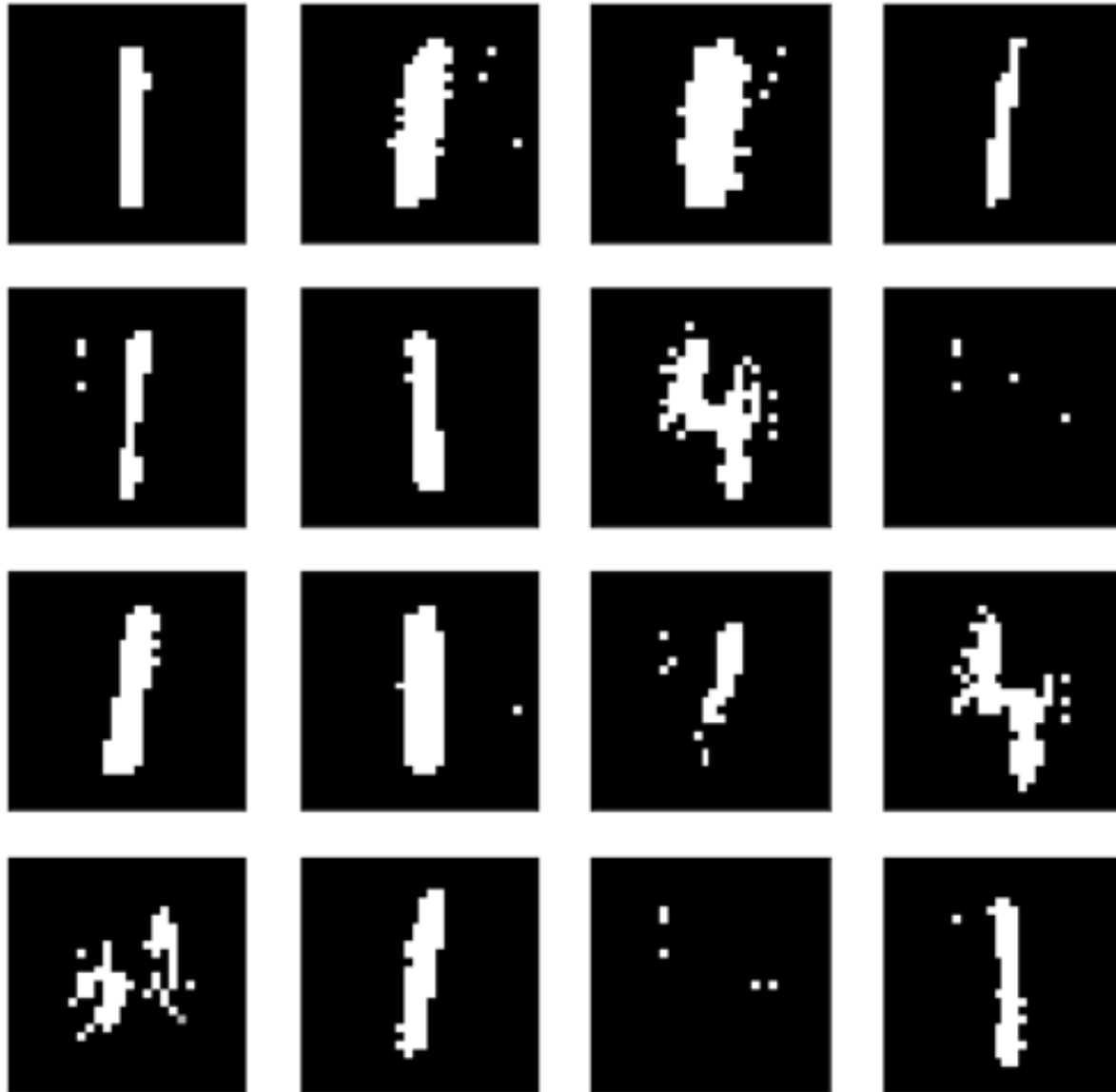
Mode Collapse

- GAN model learns to generate one type of data (e.g., only digit 1)
- Generating anything else leads to detection by discriminator
- Generator gets stuck in that local optima

➤ Q: How do we prevent/discourage mode collapse?

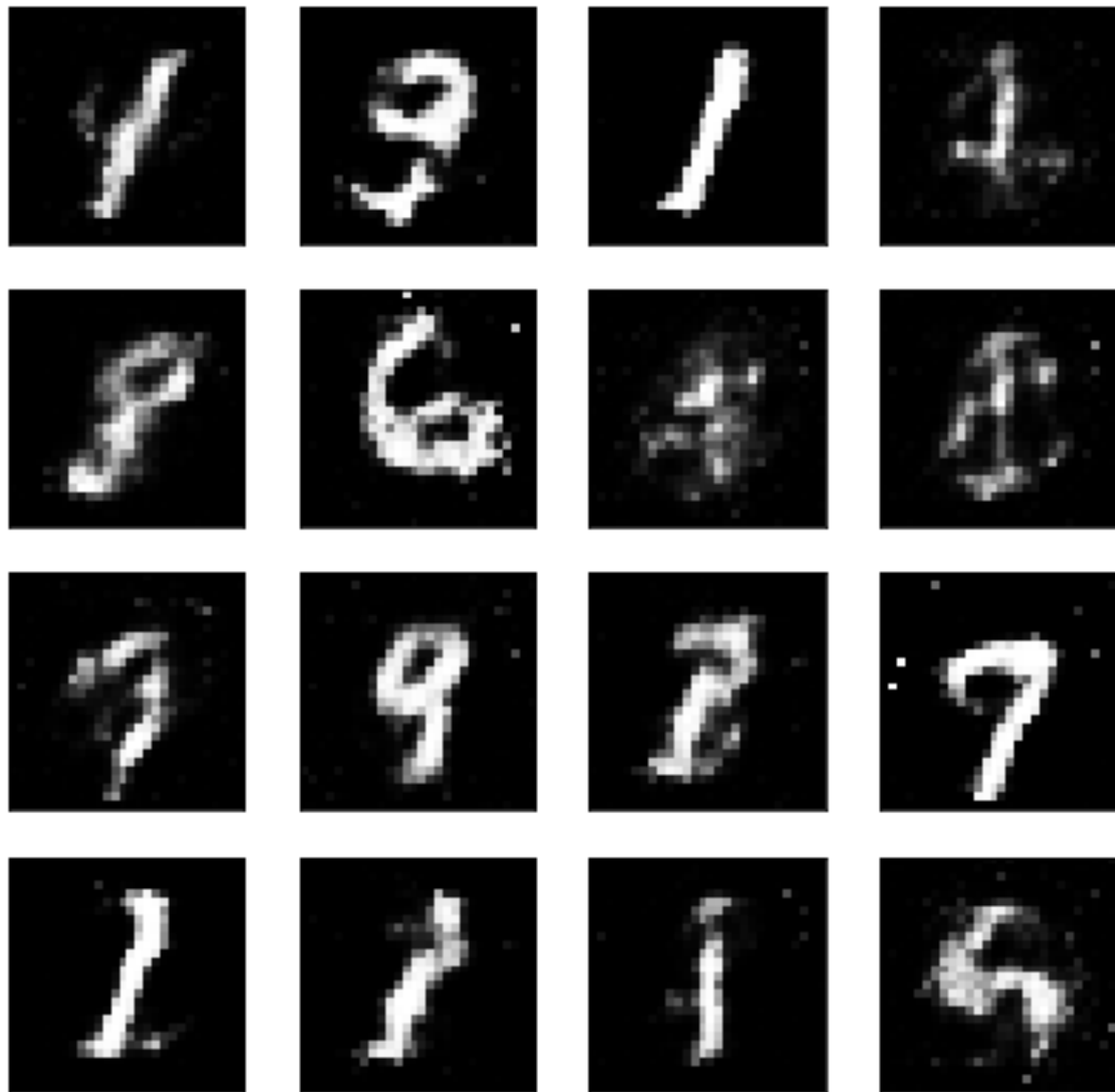
mini-batch of the fake $\rightarrow$ statistics $\Rightarrow$ low variance
+ mini-batch of the real $\Rightarrow$ statistics $\Rightarrow$ higher variance.

Generated MNIST Samples Con't



Can be improved
with more training
and tuning of your
model

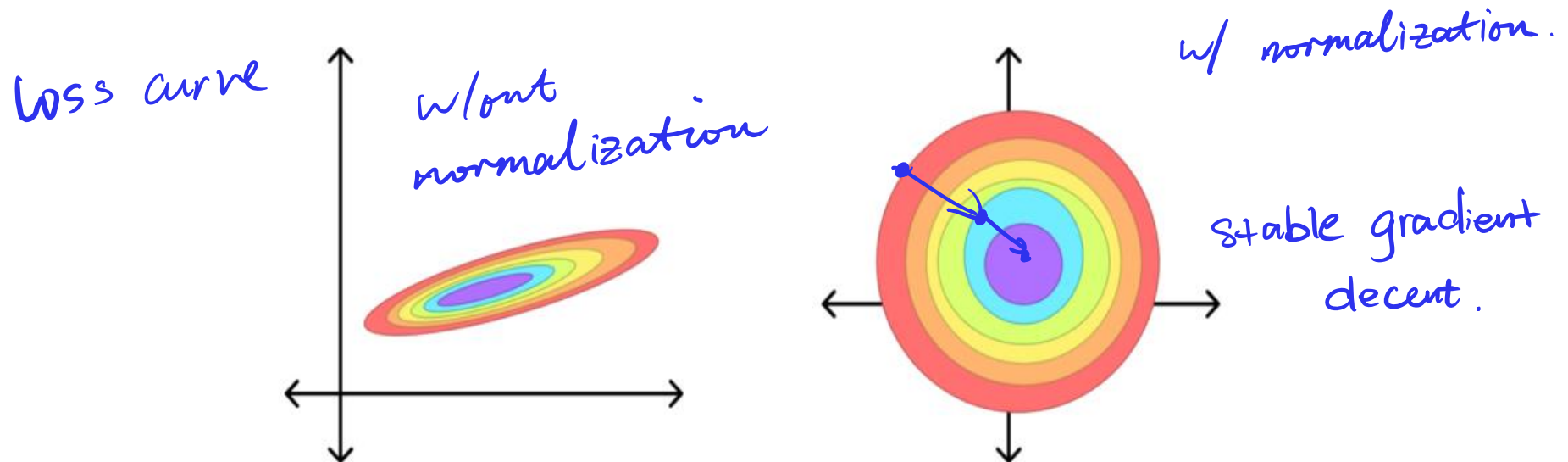
Generated MNIST Samples Con't



Can be improved
with more training
and tuning of your
model

Batch Normalization

- Like z-score normalization but applied to the hidden layers/activations to improve cost surface.



- **Training time:** normalize activations based on mini-batch statistics, and keep track of those statistics
- **Test time:** normalize activations based on saved statistics

GAN Now (Real or Fake?)



Exploring GANs

Taking GANs Further...

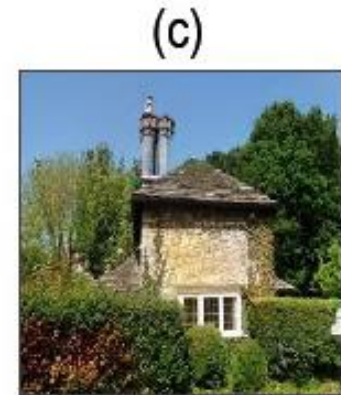
- Generating random images is impressive but has limited application.
- The following are some examples of how this novel idea of adversarial learning can be applied to real-world problems.



GAN: Greyscale to Colour

Colorization
results with
Places365.

(a) Grayscale. (b)
Original Image.
(c) Colorized with
GAN.



<https://github.com/ImagingLab/Colorizing-with-GANs>

GAN: Greyscale to Colour



Image colorized
by DeOldify:
“Interior of Miller
and Shoemaker
Soda Fountain,
1899”

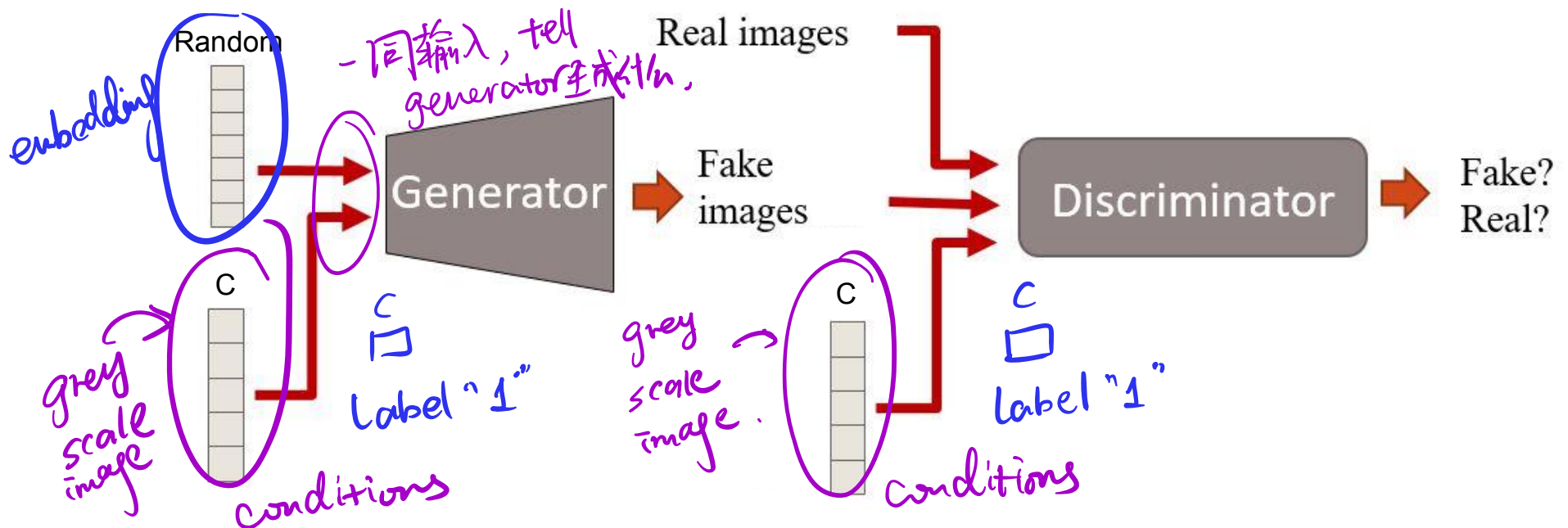


Psycho (1960)

<https://www.youtube.com/watch?v=I3UXXid04Ys>

GAN: Greyscale to Colour (version 1)

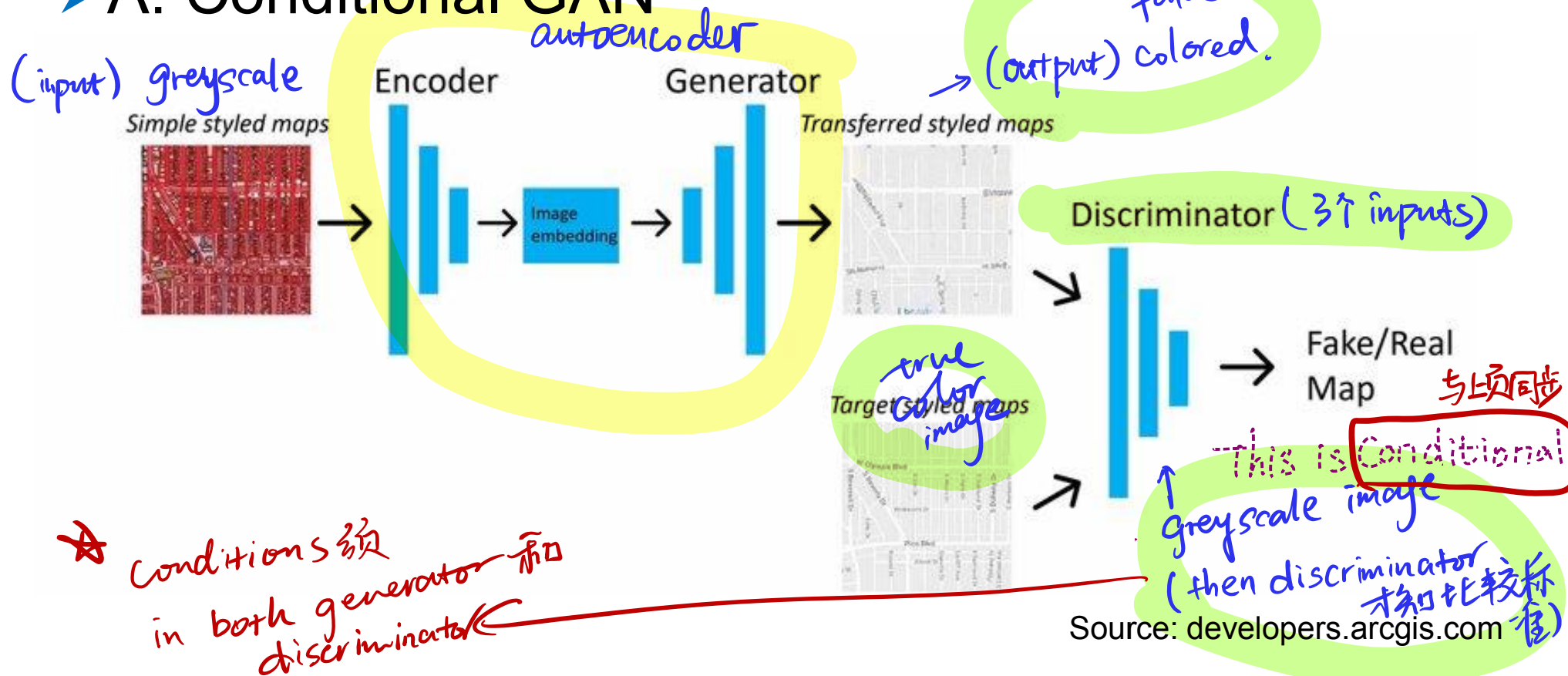
- Q: How could we use a GAN for converting greyscale images to colour?
- A: Conditional GAN



GAN: Greyscale to Colour (version 2)

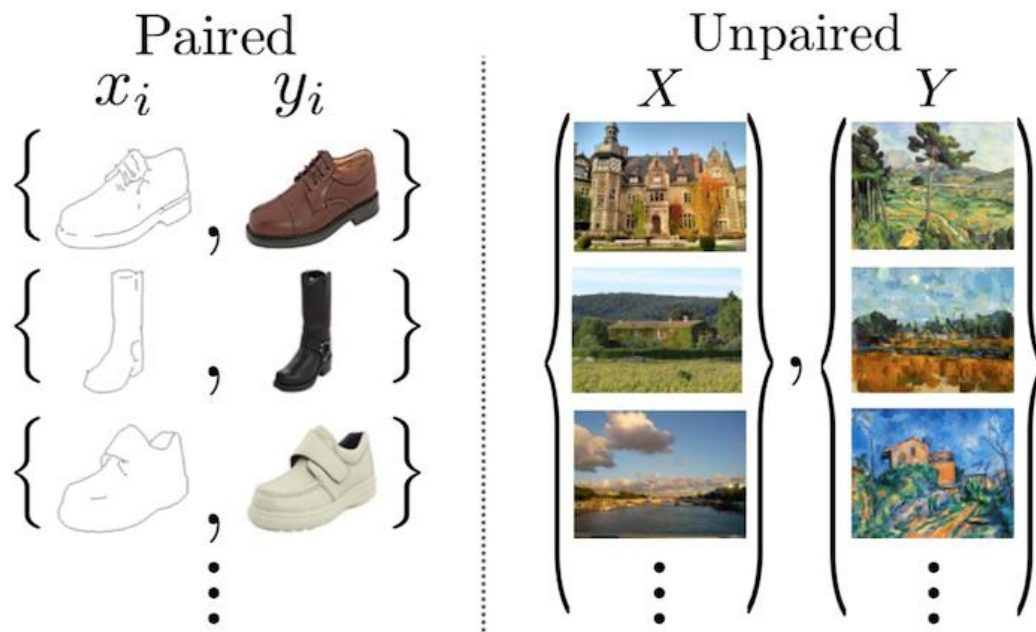
➤ Q: How could we use a GAN for converting greyscale images to colour?

➤ A: Conditional GAN



Conditional GAN with Unlabeled Data

- Q: What if we don't have paired images? Is there some way we could use the concept of GANs to learn a style transfer?

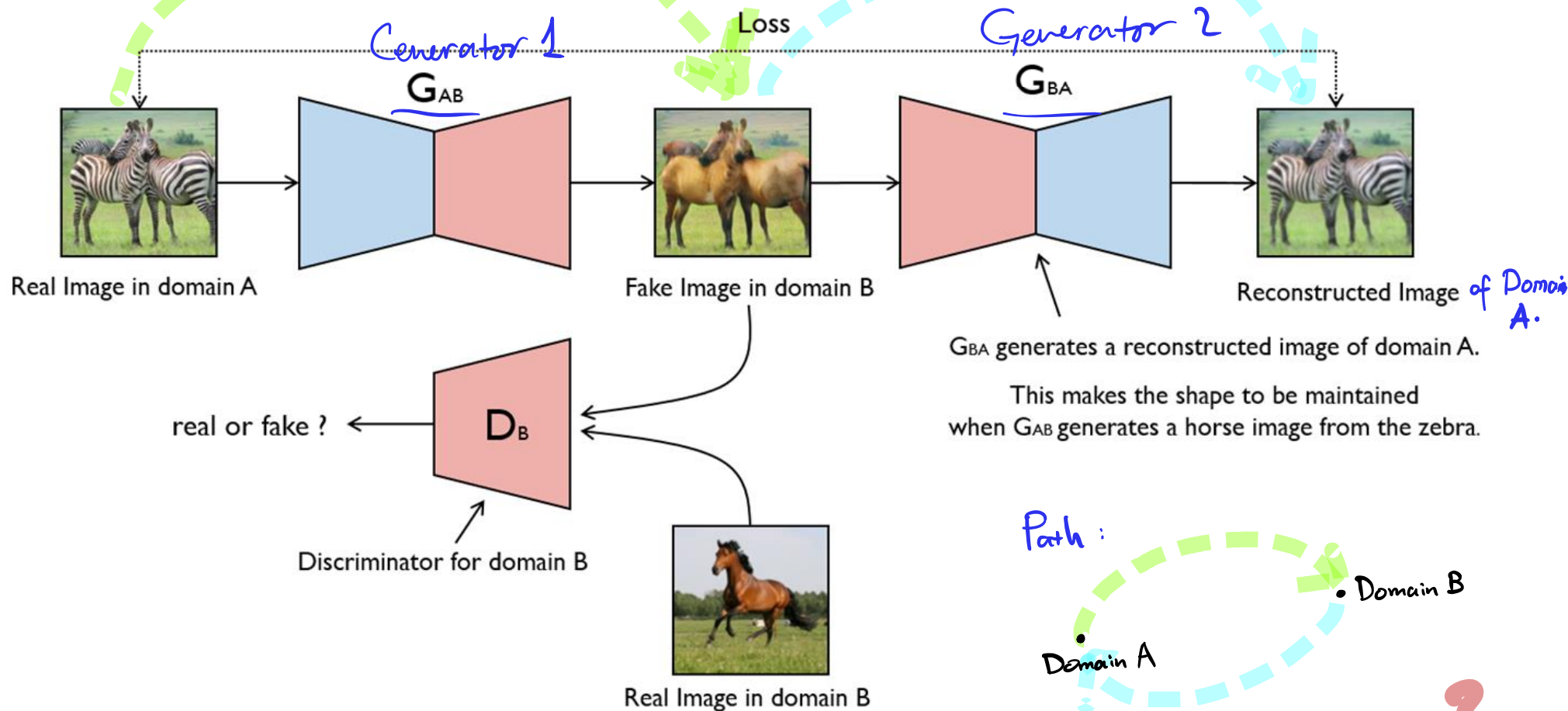


If the images are paired then we could use a conditional GAN.

Otherwise what can we do?

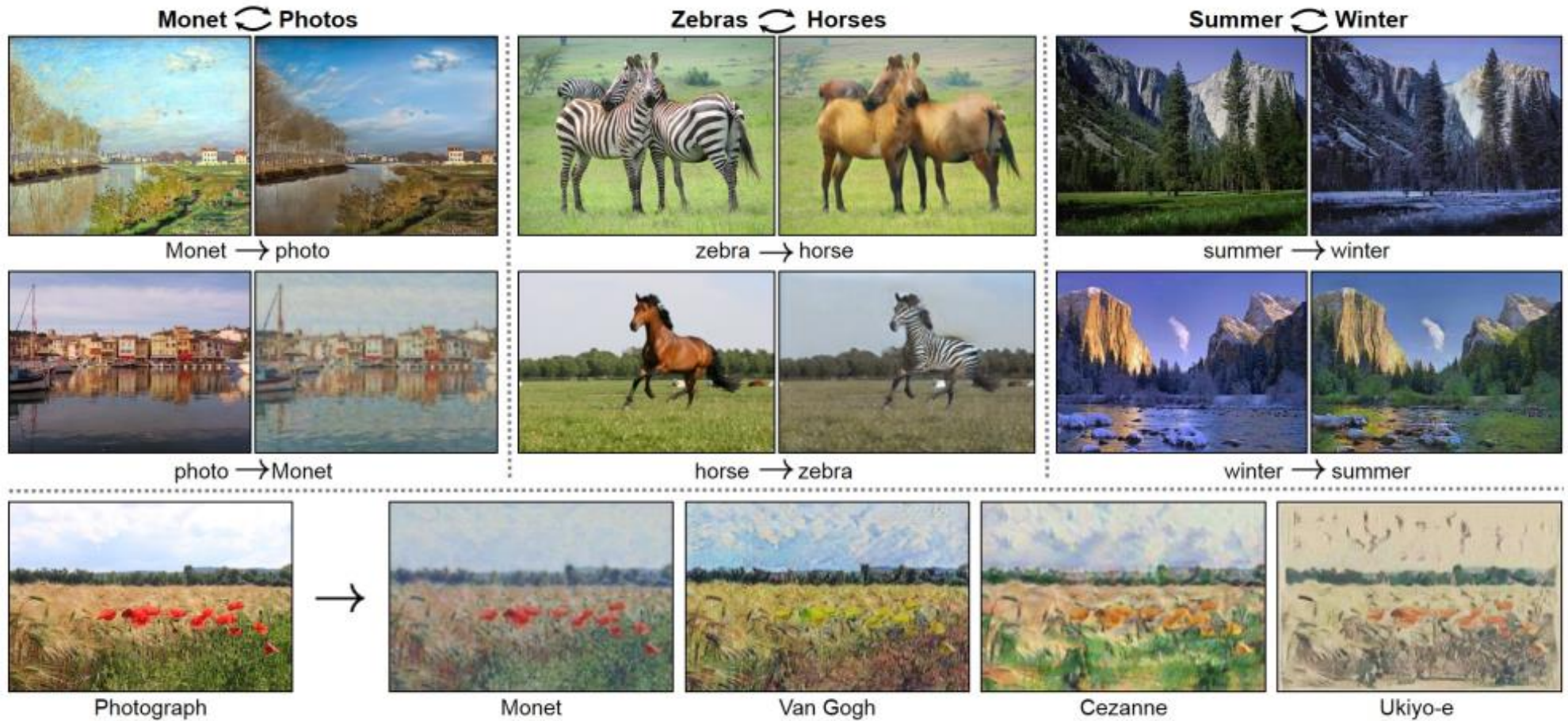
Cycle GAN

➤ This is the problem that Cycle GANs solve...



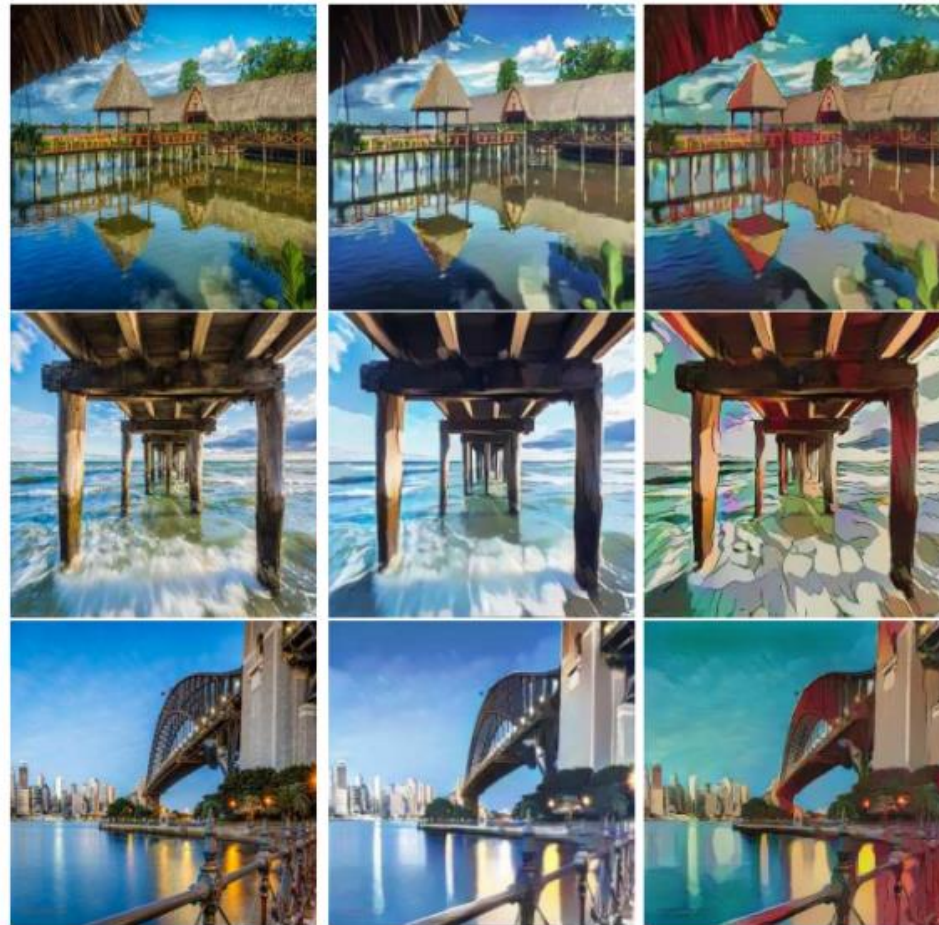
从A→B再→A, 生成的A
须与原始A相同。如果不, 重复?

Cycle GAN



<https://junyanz.github.io/CycleGAN/> (2017)

Cartoon GAN



(a) input photo

(b) Shinkai style

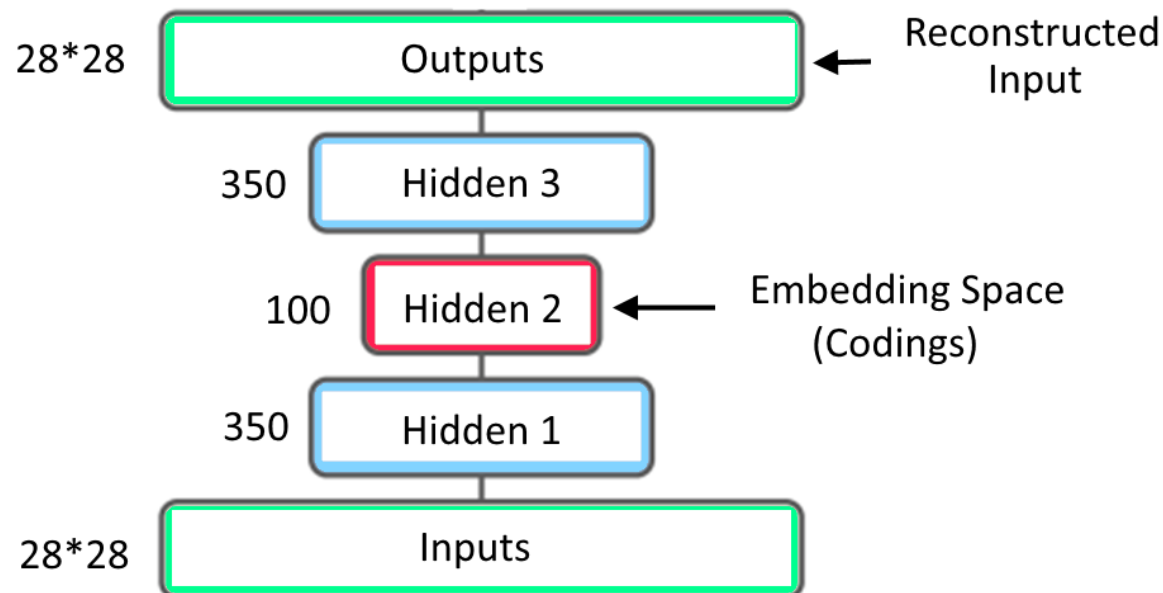
(c) Hayao style

Figure 5. Some results of different artistic styles generated by CartoonGAN. (a) Input real-world photos. (b) Makoto Shinkai style. (c) Miyazaki Hayao style.

Deep Autoencoders (Recap)

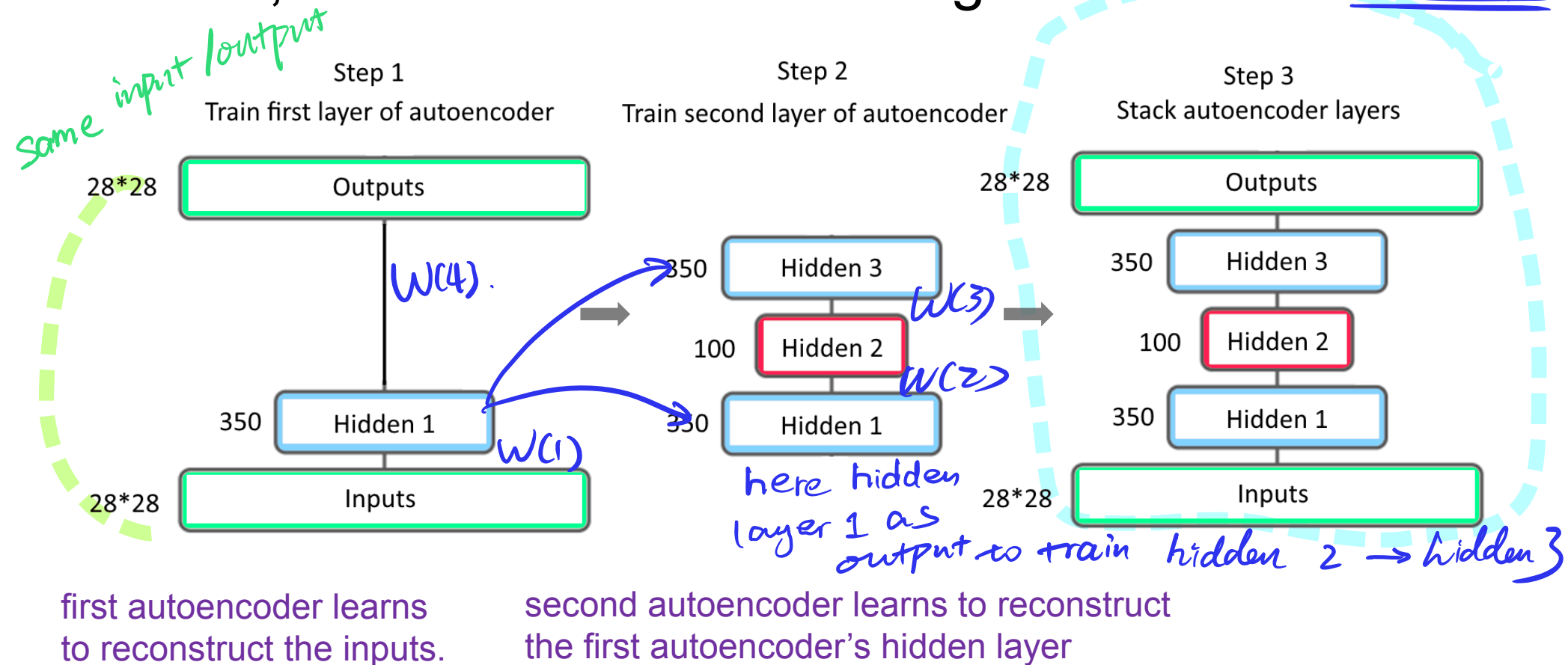
Stacked (Deep) Autoencoders

- Autoencoders can have multiple hidden layers.
- Called stacked autoencoders (or deep autoencoders)
- The architecture of a stacked autoencoder is typically symmetrical with regards to the central coding layer.
- eg.,



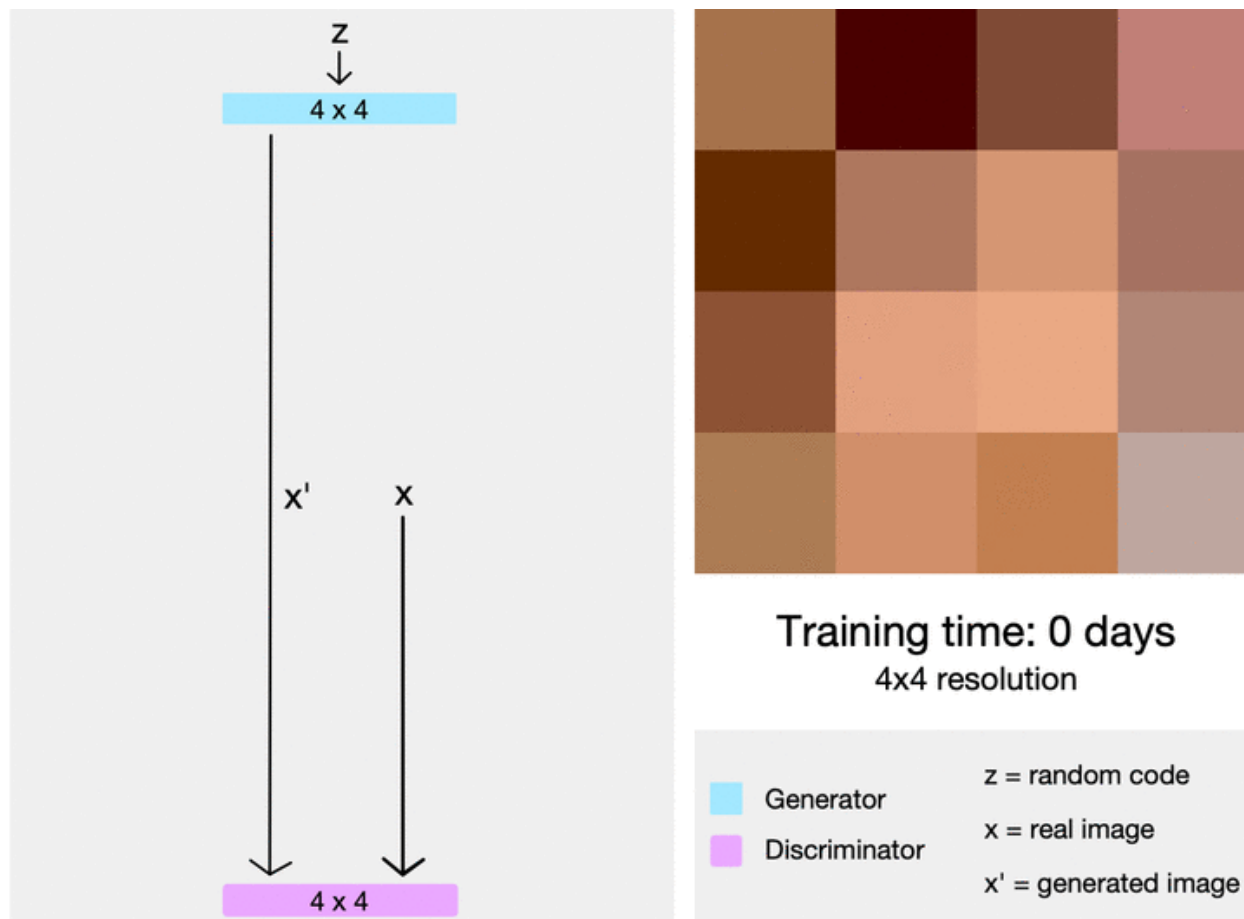
(Optional) Training Approaches

- Train entire architecture together...
- ... could also train one shallow autoencoder at a time, then stack all of them together



Progressive Growing of GANs

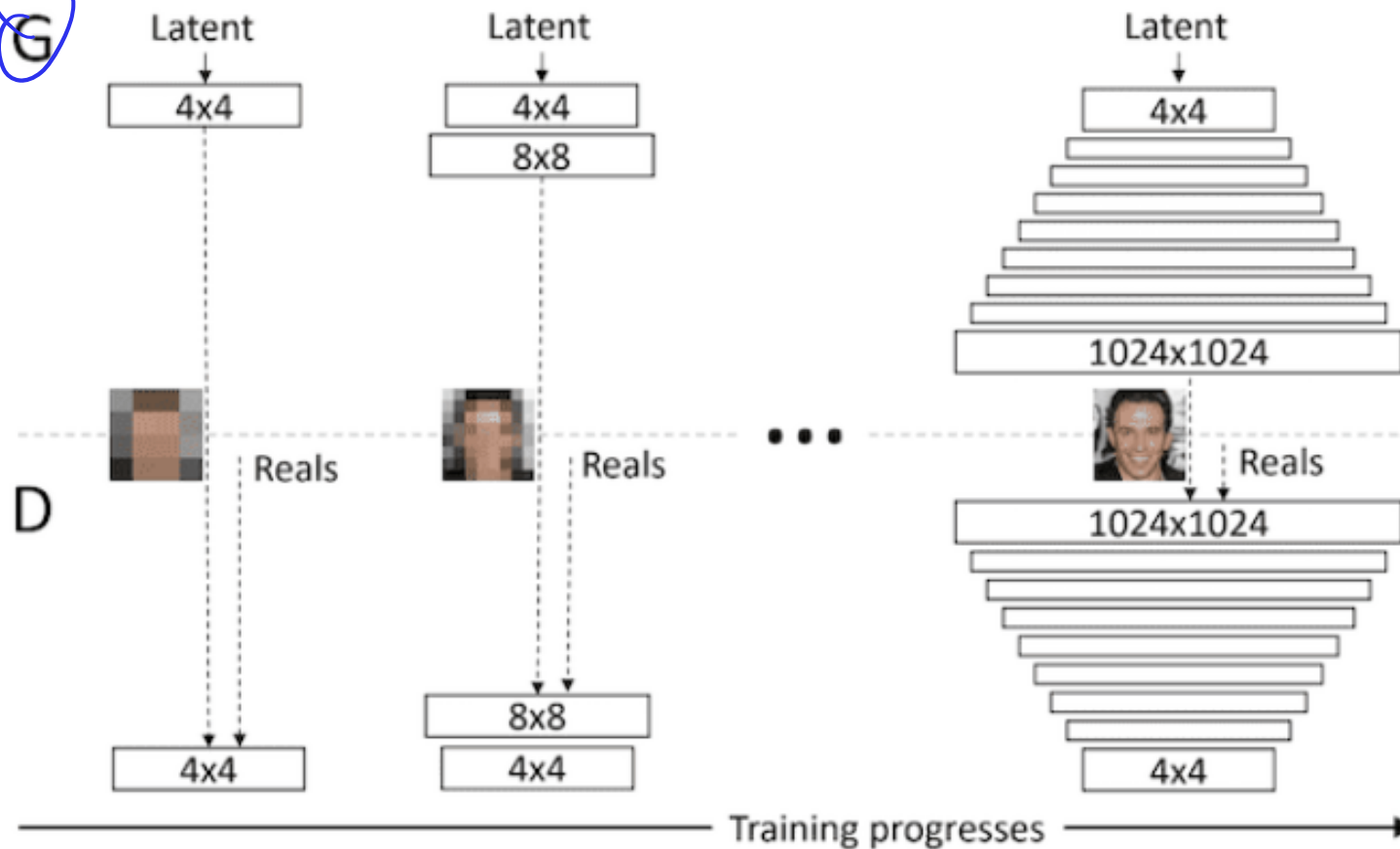
➤ Incremental training based on resolution.



somewhat
similar to
training
autoencoder
in stages

Progressive Growing of GANs

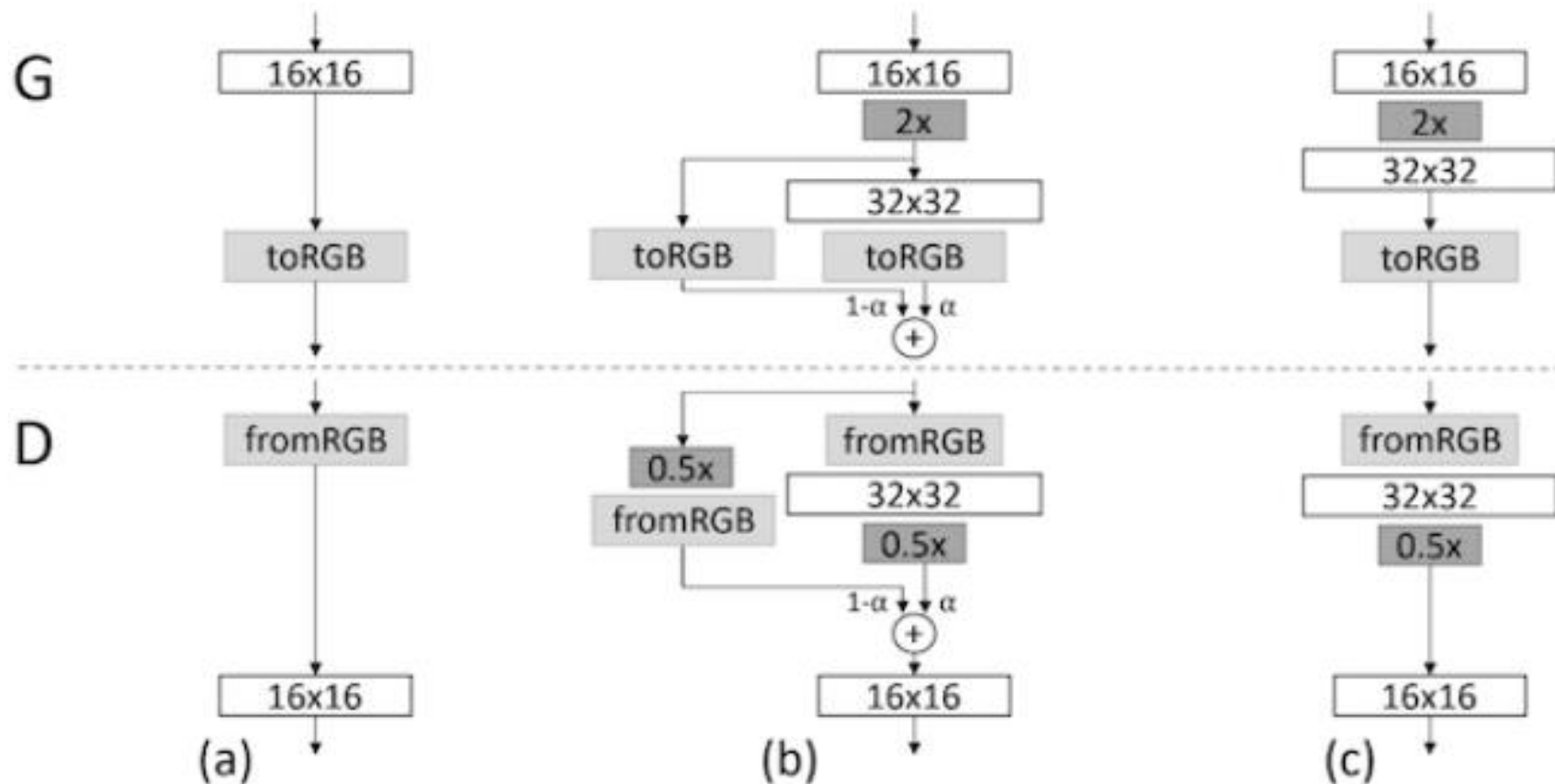
→ start by low pixel by generator, the discriminative keeps evaluate.
→ Incremental training based on resolution.



until get the final high-quality images.

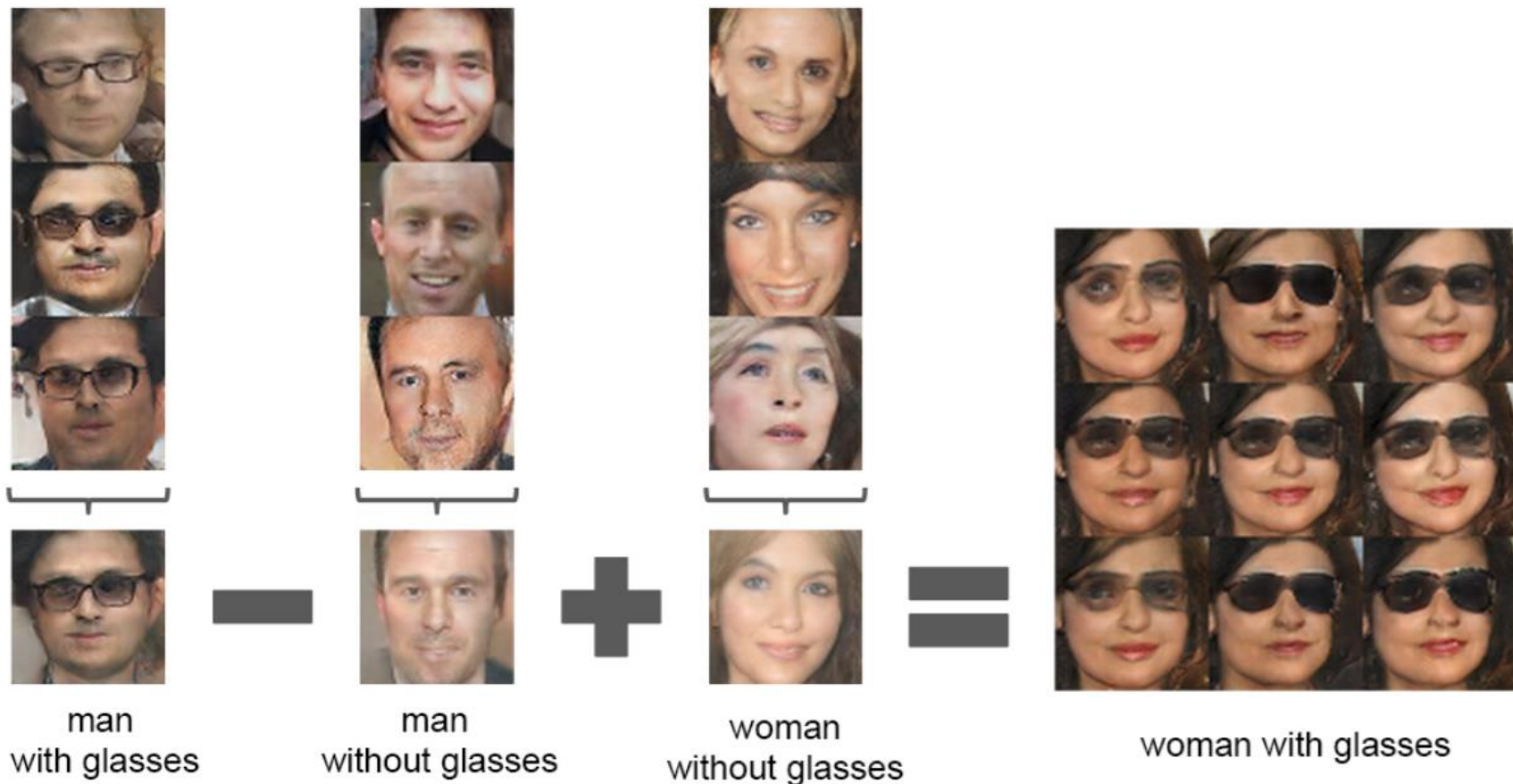
Progressive Growing of GANs

- Incremental training based on resolution.



Using Embedding Spaces

- Learn a good embedding space to guide image generation.



Using Embedding Spaces

- Learn a good embedding space to guide image generation.



Using Embedding Spaces



Source: [Yildirim et al., 2019](#)

Using Embedding Spaces

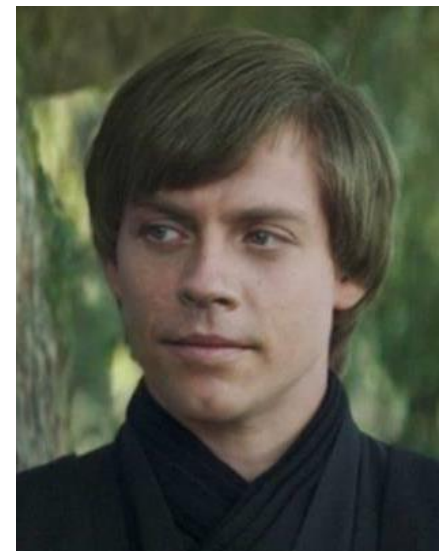
- Examples using StyleGAN2 (state-of-the-art)



<https://www.youtube.com/watch?v=9QuDh3W3IOY>

Summary

- Replace loss function with adversarial 2-player game. Allows the network to learn an appropriate loss function.
- Achieves impressive results! Many useful applications (e.g., Disney's Star Wars franchise)
- Difficult to guide the training process!



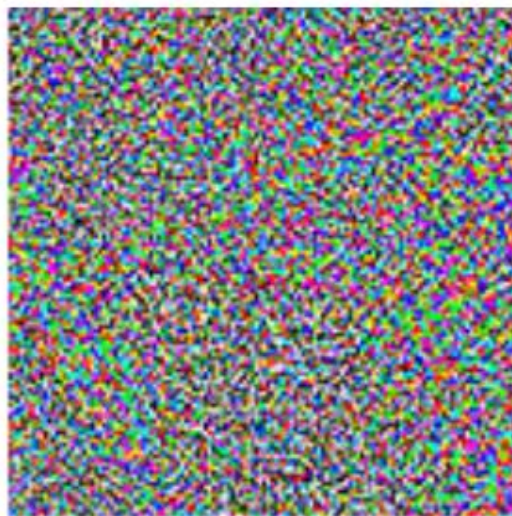
Source: Disney / Lucasfilm
(The Book of Boba Fett)

Adversarial Examples

What is this a picture of?



+ ϵ



=



"panda"

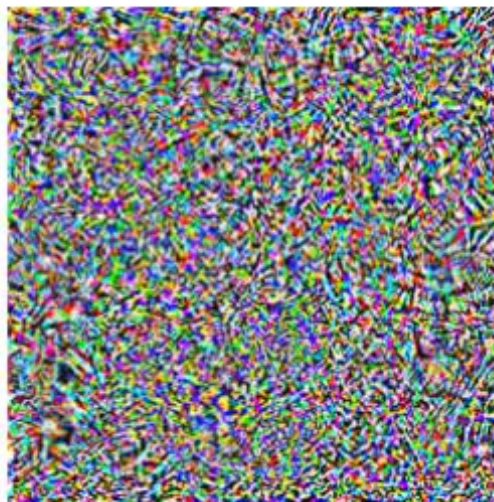
57.7% confidence

What is this a picture of?

“pig”



+ 0.005 x



=



Adversarial Attacks

- **Goal:** Choose a small perturbation ε on an image x so that a neural network f misclassifies $x + \varepsilon$.
- **Approach:** Use the same optimization process to choose ε to minimize the probability that

$$f(x + \varepsilon) = \textit{correctclass}$$

- We are treating ε as the **parameters**.

Targeted vs Non-Targeted Attacks

➤ Non-targeted attack

- Minimize the probability that
$$f(x + \epsilon) = \text{correctclass}$$

➤ Targeted attack

- Maximize the probability that
$$f(x + \epsilon) = \text{targetclass}$$

like input model 1

*but want to generate
(target) a model 5*

White-Box vs Black-Box Attacks

➤ **White-Box Attack**

- Assumes that the model is known
- We need to know the architectures and weights of f to optimize ε

➤ **Black-Box Attack**

- Don't know the architectures and weights of f to optimize ε
- Substitute model mimicking target model with known, differentiable function
- adversarial attacks often transfer across models!

Example: Python Code

Q: What type of adversarial attack is this?

```
def create_adversarial_example(model, image, target_label):  
    noise = torch.randn(1, 28, 28)  
    noise.requires_grad = True  
  
    optimizer = optim.Adam([noise], lr=0.01, weight_decay=1)  
    criterion = nn.CrossEntropyLoss()  
  
    for i in range(1000):  
        adv_image = torch.clamp(image + noise, 0, 1)  
        out = model(adv_image.unsqueeze(0))  
        loss = criterion(out, torch.Tensor([target_label]).long())  
        loss.backward()  
        optimizer.step()  
        optimizer.zero_grad()
```

Pretrained
CNN model

Adversarial Sample Code

Example: 3D Objects

https://www.youtube.com/watch?v=piYnd_wYIT8



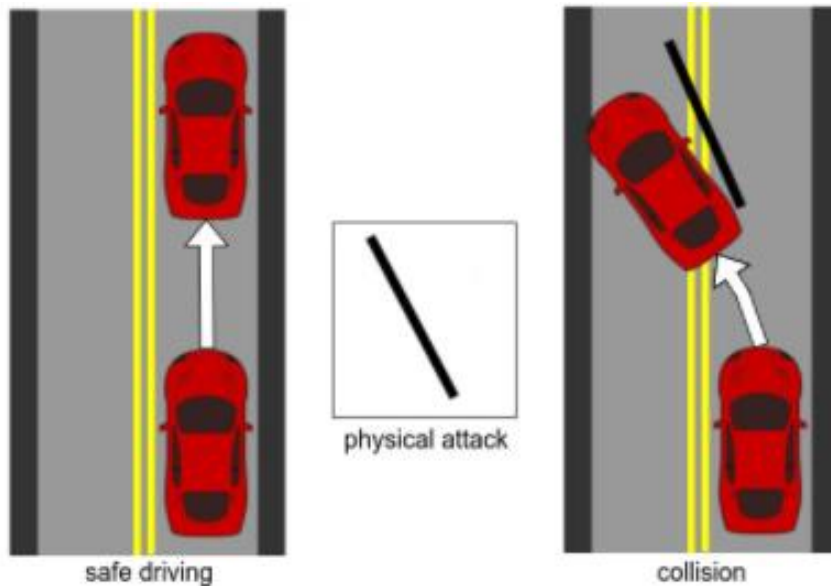
Example: Printed Pictures

<https://www.youtube.com/watch?v=MlbFvK2S9g8&feature=youtu.be>

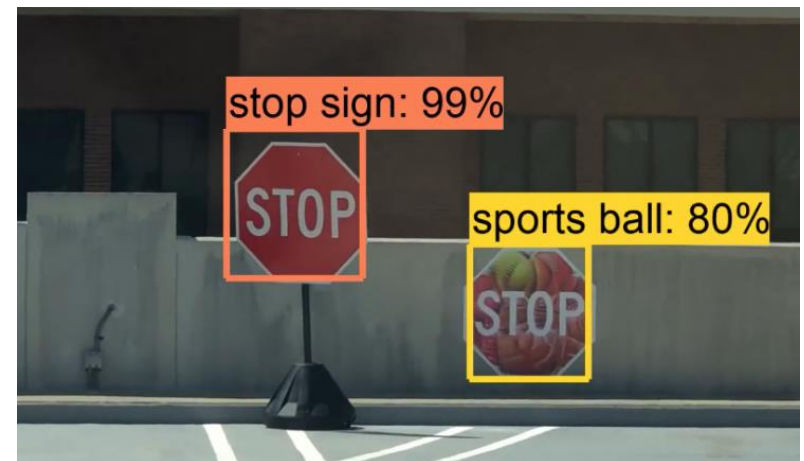


Example: Traffic Signs

➤ Active area of research



Source: Boloor et al., 2019



Failed Defenses

There have been many attempts to modify the architectures or training to defend against attacks with no success, for example:

- Adding noise at test time
- Averaging many models
- Weight decay
- Adding noise at training time
- Adding adversarial noise at training time
- Dropout

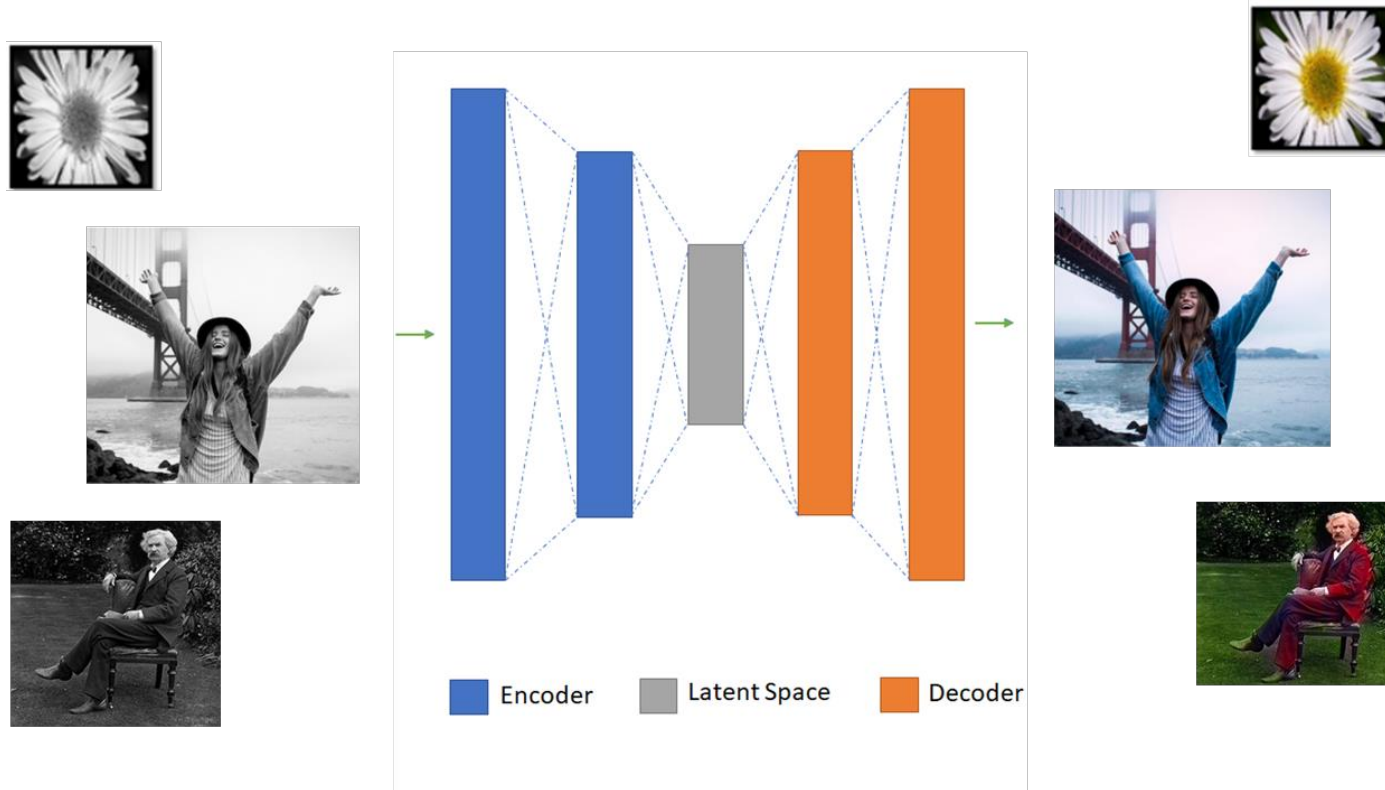
Week 7 - Announcements

Projects: Progress Report

- Progress reports due Tuesday, Nov 11 at 11:00pm
- Progress report is a check-in to show that you have made progress towards the completion of the project. Your team should have:
 - Obtained all or most of the data
 - Come up with a reasonable model
 - Produced at least one promising result
 - Provide a convincing argument for the feasibility of completing the project within the time available
- The progress report will be assessed **out of 30 points** based on: Data Collected, Results Achieved, and the clarity/readability of your Jupyter Notebook submission.

Assignment 3: Image Colourization

➤ Image to Image Translation



➤ Due in three weeks (11pm on Tuesday, Nov 4th)

Next Time

Homework

- Review A3 Sample Code

Q/A Support Session

- Assignment 3, Project Discussions

Midterm (Wednesday, Oct 22 lecture time)

Reading Week

Lecture Week 8

- Word Embeddings
- Recurrent Neural Networks (Part 1)

Have a Happy Halloween!



Images generated by StyleGAN2!

